DAT62-1

Introduction to Generative AI and LLMs

Training, ecosystem, RAG, evaluation, and deployment risk

Understand generative AI as a system, not just a tool to call.

Albert School — 22 hours

Contents

1 The LLM training pipeline

A large language model is built in stages. Each stage changes what the model does, and the behaviour of the deployed model is the cumulative result of all of them.

Definition 1 **Pretraining** fits the model to predict the next token on a very large corpus of text. The only objective is next-token prediction. This stage gives the model its broad knowledge of language and facts, but it produces a model that continues text rather than one that follows instructions or answers questions directly.

Definition 2 **Instruction tuning** (supervised fine-tuning) continues training the pretrained model on examples of instructions paired with desired responses. This stage changes the model's behaviour from continuing text to following instructions and responding in the expected format.

Definition 3 **Reinforcement learning from human feedback** (RLHF) further adjusts the model using human preferences between candidate responses. A reward model is trained to predict which response humans prefer, and the language model is optimised against that reward. This stage aligns the model's outputs with human preferences for helpfulness, harmlessness, and tone.

The three stages are sequential: pretraining supplies knowledge and language, instruction tuning makes the model usable as an assistant, and RLHF shapes the style and safety of its responses.

2 The LLM ecosystem

The same underlying model can be deployed at different levels of autonomy. The distinctions below matter because they determine what a system can do and what can go wrong.

Definition 4 A **model** (LLM) is the trained network itself: it maps an input context to a distribution over the next token and generates text. On its own it has no memory between calls and no ability to take actions. Example: Claude as a model accessed through the API.

Definition 5 A **conversational agent** wraps a model in an interface that maintains the conversation and presents the model as a chat assistant. Example: the Claude chat application.

Definition 6 An **agentic system** wraps a model so that it can take actions — call tools, read and write files, run commands — in a loop, pursuing a goal across multiple steps rather than producing a single reply. Example: Claude Code.

Definition 7 The **Model Context Protocol** (MCP) is a standard interface that connects a model to external tools and data sources. It matters for system architecture because it provides a uniform integration layer: tools and data sources implemented once against MCP can be reused across different models and applications, instead of each integration being built ad hoc.

3 RAG architecture

Definition 8 A model's knowledge is fixed at training time and limited to what was in its training data. It does not know facts that are private, that postdate training, or that are too specific to have been learned. **Retrieval-augmented generation** (RAG) addresses this by retrieving relevant external documents at query time and supplying them to the model as context, so the model answers from provided material rather than from its parameters alone.

A RAG system has four stages: chunk the source documents, embed the chunks, retrieve the chunks relevant to a query, and generate an answer from them.

Definition 9 **Chunking** splits source documents into smaller passages. Chunks must be small enough to embed and to fit in the model's context, but large enough to remain self-contained and meaningful. Common strategies split by a fixed token length, by sentence or paragraph boundaries, and use overlap between consecutive chunks so that information spanning a boundary is not lost.

Definition 10 **Embedding-based retrieval** represents each chunk as a vector produced by an embedding model, so that semantically similar texts have nearby vectors. A query is embedded the same way, and the chunks whose vectors are closest to the query vector (by cosine similarity) are retrieved.

Definition 11 A **vector store** is a database that holds the chunk embeddings and supports efficient nearest-neighbour search over them, returning the most similar chunks for a query vector. FAISS is a common example.

4 RAG implementation

A minimal RAG pipeline is assembled from the stages above.

Definition 12 The implementation steps are:
1. **Chunk** the documents into passages.
2. **Embed** each chunk with a sentence transformer (a sentence embedding model).
3. **Store** the chunks and their embeddings in a vector store such as FAISS.

4. **Query**: embed the user's question and retrieve the nearest chunks.
5. **Generate**: place the retrieved chunks in the prompt as context and have the LLM produce an answer grounded in them.

The retrieved chunks are inserted into the prompt together with the question, so the model conditions its answer on the retrieved material.

5 Evaluating generative AI outputs

Generative outputs have no single correct answer, so evaluation combines qualitative and quantitative methods.

Definition 13 A **qualitative rubric** defines explicit criteria — for example relevance, correctness, completeness, and tone — and rates each output against them. The rubric makes evaluation systematic and repeatable rather than impressionistic.

Definition 14 **BLEU** and **ROUGE** are quantitative metrics that score a generated text against one or more reference texts by measuring overlap of word sequences (n-grams). BLEU is precision-oriented and was designed for machine translation; ROUGE is recall-oriented and was designed for summarisation. Both require reference texts.

Definition 15 **LLM-as-judge** uses a separate language model, prompted with explicit criteria, to score or compare candidate outputs. It scales qualitative judgement to many examples without a human rating each one, and does not require reference texts.

Definition 16 A **comparative evaluation** applies the same rubric or metric to two or more candidates (for example two prompts or two systems) under identical conditions and reports the results side by side, so the better candidate can be identified systematically.

6 Failure modes and deployment risks

LLM-based systems have characteristic failure modes. Each must be identified and paired with at least one mitigation before deployment.

Definition 17 **Hallucination** is the generation of fluent but false or unsupported content. It ranges across fabricated facts, incorrect reasoning, and invented citations or sources. Mitigation: ground the model in retrieved sources (RAG), require citations, and verify factual claims against authoritative data.

Definition 18 **Data privacy** risk is the exposure of sensitive data, for instance by sending private information to a third-party model provider or by including it in prompts that are logged. Mitigation: minimise and redact sensitive data, control what is sent to external providers, and use deployments with appropriate data handling guarantees.

Definition 19 **Bias amplification** is the model reproducing or strengthening biases present in its training data. Mitigation: evaluate outputs across affected groups, and constrain or post-process outputs in sensitive applications.

Definition 20 **Prompt injection** is an attack in which malicious instructions placed in the input or in retrieved content cause the model to ignore its intended instructions. Mitigation: separate trusted instructions from untrusted content, constrain the model's tools and permissions, and validate inputs.

Definition 21 **Data exfiltration** is an attack in which a model is manipulated into leaking private data it has access to, for example through tool calls or crafted outputs. Mitigation: restrict the model's access and tool permissions, and validate or filter outputs before they leave the system.

7 When to use generative AI

Definition 22 Deciding whether an LLM-based solution is appropriate is a business-case assessment that weighs **cost**, **quality**, and **latency**. LLM calls cost money per token and add latency; a larger or more capable model raises quality but increases both cost and latency. The choice balances these against the value the task delivers and the accuracy it requires.

An LLM-based solution is justified when the task needs flexible language understanding or generation that simpler, cheaper, or more reliable methods cannot provide, and when its cost and latency are acceptable for the value delivered. When a deterministic method or a smaller model meets the requirement, the LLM is wasteful.

Exercises

1. Describe the three stages of the LLM training pipeline (pretraining, instruction tuning, RLHF) and state what each one changes about the model's behaviour.
2. Given concrete systems, classify each as a model, a conversational agent, or an agentic system, and justify the classification. Explain what MCP is and why it matters for system architecture.
3. Explain why an LLM needs external knowledge and how RAG supplies it. Describe the four stages of the RAG architecture.
4. Implement a minimal RAG pipeline: chunk a set of documents, embed the chunks with a sentence transformer, store them in FAISS, retrieve for a query, and generate an answer grounded in the retrieved chunks.
5. Evaluate generative outputs both with a qualitative rubric and with a quantitative metric (BLEU, ROUGE, or LLM-as-judge), and produce a comparative evaluation of two candidates.
6. For each of hallucination, data privacy, bias amplification, prompt injection, and data exfiltration, describe the risk and give at least one mitigation.
7. Assess a business use case for generative AI: evaluate the cost, quality, and latency tradeoffs and justify whether an LLM-based solution is appropriate.